AI Resume Analyzer & Recommendation

Subject - Ai

Name of Student	Roll No.
Om Shelke	25102B2011
Aditya Vibhute	25102B2002

Supervisor

Prof. Mahesh Khandke

VIT | Vidyalankar Institute of Technology

Accredited A+ by NAAC

Vidyalankar Institute of Technology

(AY 2026-27)

Department of Computer Engineering

CERTIFICATE

This is to certify that the Mini Project entitled **"AI Resume Analyzer & Recommendation" for the Subject Artificial Intelligence (TE CMPN, Sem 5)** is a bonafide work of **Om Shelke (25102B2011) and Aditya Vibhute (25102B2002).**

Prof. Mahesh Khandke
Supervisor

Contents

Abstract ..... **ii**

Acknowledgments ..... **iii**

List of Abbreviations ..... **iv**

List of Figures ..... **v**

List of Tables ..... **vi**

List of Symbols ..... **vii**

1 Introduction **1**

1.1 Problem Statement & Objectives 1

2 Literature Survey **4**

2.1 Survey of Existing/Similar System 4

3 Proposed System **6**

3.1 Introduction 6

3.2 Architecture/ Framework 8

3.3 Algorithm and Process Design 9

3.4 Details of Hardware & Software 13

3.5 Experiment and Results 15

3.6 Conclusion and Future Work 22

References **25**

Abstract

Recruiters receive hundreds of applications for a single technology opening, while candidates rarely learn why a resume was rejected or which skills it lacked. This project presents **AI Resume Analyzer & Recommendation**, a full-stack web application that treats resume screening as a data-mining problem. Unstructured data (resumes in PDF or DOCX format and job descriptions) is extracted, cleaned, converted into numeric features and compared with similarity measures to produce ranked, explainable results.

The system parses resumes with pdfplumber and python-docx, detects sections and extracts structured fields with regular expressions, and identifies skills using a curated taxonomy of seven skill domains with alias resolution. A transparent six-component weighted model (skills, experience, projects, education, completeness and keywords) gives each resume a score from 0 to 100 together with strengths, weaknesses and recommendations. Jobs are recommended with a hybrid model that combines TF-IDF cosine similarity (weight 0.55) with Jaccard overlap of skill sets (weight 0.45), followed by a skill-gap analysis that lists the skills a target job requires but the resume does not contain.

The application is built with a React and Vite frontend, a Flask REST API, scikit-learn and NLTK for the machine-learning layer, and SQLite with SQLAlchemy for persistence. In the test run reported here, 14 skills were detected across 5 categories, the overall resume score was 67.9 out of 100 and the ATS-compatibility score was 70.6%. The approach needs no labelled training data or GPU and keeps every score explainable. Its limitations (layout-sensitive parsing, no OCR, a static job dataset and empirically chosen weights) are discussed together with future work such as embedding-based matching and fairness auditing.

Keywords: Resume Screening, Job Recommendation, TF-IDF, Cosine Similarity, Jaccard Index, Skill-Gap Analysis, Text Mining, ATS.

Acknowledgments

We would like to express our sincere gratitude to our guide and supervisor, **Prof. Shweta Jaiswar**, for her constant guidance, encouragement and valuable suggestions throughout the course of this mini project for the subject Data Warehouse and Mining.

We are thankful to the Head of the Department of Computer Engineering and to all the faculty members of the department for providing the facilities and the academic environment needed to complete this work.

We also thank **Vidyalankar Institute of Technology** for giving us the opportunity to apply the concepts learned in class to a practical, real-world problem. Finally, we thank our families and friends for their support and motivation.

Aditya Vibhute (25102B2002)
Om Shelke (25102B2011)

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
BERT	Bidirectional Encoder Representations from Transformers
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DOCX	Microsoft Word Open XML Document
DWM	Data Warehouse and Mining
ETL	Extract, Transform, Load
HR	Human Resources
JSON	JavaScript Object Notation
LTR	Learning to Rank
ML	Machine Learning
NER	Named Entity Recognition
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
SQL	Structured Query Language
TF-IDF	Term Frequency - Inverse Document Frequency
UI	User Interface

List of Figures

Figure No.	Title	Page
3.1	System architecture	8
3.2	Landing page with the platform introduction and a sample score card	15
3.3	Landing page: comprehensive AI and NLP features	16
3.4	Landing page: how the AI system works (four-step workflow)	16
3.5	Landing page: core technologies and call to action	16
3.6	Dashboard: overall score, skills detected, key skill gaps and candidate profile	17
3.7	Resume Analysis: analyser score and weighted scoring formula	17
3.8	Resume Analysis: candidate identification and extracted technical competencies	18
3.9	Resume Analysis: education, work experience, projects and ATS compatibility breakdown	18
3.10	Resume Analysis: quality metrics, strengths, identified gaps and actionable tips	19
3.11	Job Matching: job description input and match calculation	19
3.12	Job Recommendations: AI-ranked jobs with match percentages and filters	19
3.13	Resume Improvement: section-wise quality audit and suggestions	20
3.14	Analysis History: previous analyses with scores and top job match	20
3.15	About Project: problem statement and system architecture	20
3.16	About Project: end-to-end workflow and mathematical formulations	21

List of Tables

Table No.	and Title	Page
1.1	Mapping of objectives to implementation	3
2.1	Comparison of existing approaches	5
3.1	Mapping of project stages to DWM concepts	6
3.2	Fields of the job dataset	7
3.3	Skill taxonomy categories	7
3.4	Layered architecture	9
3.5	Database entities	9
3.6	Scoring components	11
3.7	Hardware requirements	14
3.8	Backend technologies	14
3.9	Machine learning and NLP libraries	14
3.10	Parsing and document libraries	14
3.11	Frontend technologies	15
3.12	Sample scoring breakdown (scenario profiles)	21

List of Symbols

Symbol	Meaning
A	Set of skills extracted from the resume
B	Set of skills required by a job
$\|A \cap B\|$	Number of skills common to A and B (intersection)
$\|A \cup B\|$	Number of distinct skills in A and B together (union)
J(A, B)	Jaccard index of skill sets A and B
a, b	TF-IDF vectors of the resume and of a job
$a \cdot b$	Dot product of vectors a and b
$\|\|a\|\|$	Euclidean norm (length) of vector a
cosine(a, b)	Cosine similarity of vectors a and b
tf	Term frequency of a word in a document
Hybrid	Combined similarity score 0.55 × Cosine + 0.45 × Jaccard
Overall	Weighted resume score on a 0-100 scale

1 Introduction

1.1 Problem Statement & Objectives

1.1.1 Background

Online job portals have made it easy for candidates to apply to many positions with a single click. As a result, a single job posting in the technology sector can attract hundreds or even thousands of applications. Recruiters must shortlist candidates quickly, and most organisations rely on Applicant Tracking Systems (ATS) that filter resumes by keywords, structure and formatting before a human ever reads them. For the candidate, this process is largely a black box: a resume is rejected without explanation, and it is unclear which skills or sections were missing.

Natural Language Processing (NLP) and classical data-mining techniques such as TF-IDF vectorisation and set-similarity measures provide a practical way to automate parts of this process. They are fast, explainable, require no labelled training data, and can run on ordinary hardware, which makes them well suited to an academic project and to small organisations.

1.1.2 Problem Statement

In today's competitive job market, the recruitment process is overwhelmed by the volume of applications. Hiring managers often spend a large amount of time manually screening resumes for a single opening, which makes the process slow, subjective and error-prone. Simultaneously, job seekers struggle to understand how well their resume aligns with a specific job description, which makes it difficult to optimise their applications. This project builds an AI-powered Resume Analyzer & Recommendation system that automates resume analysis, scores candidates objectively and recommends the most compatible jobs based on skill matching.

1.1.3 Key Challenges

Manual bottleneck. Recruiters spend a large share of their time on repetitive screening work, which delays hiring and takes time away from interviewing and evaluating candidates.

Inconsistency. Human fatigue and unconscious bias can lead to different evaluations of equally qualified candidates.

Skill-gap blindness. Candidates are often unaware of the specific skills they lack for their desired roles.

ATS failures. Qualified candidates can be filtered out because of poor formatting, missing sections or the absence of relevant keywords.

Poor job discovery. Job seekers lack tools that recommend jobs based on their actual skill set rather than on a keyword search alone.

1.1.4 Scope of the Project

The project focuses on technology-sector roles (for example Data Scientist, Backend Developer, Frontend Developer, AI/ML Engineer). The system accepts English-language resumes in PDF or DOCX format of up to 10 MB. It works as a decision-support tool for candidates and, in future versions, for recruiters. It does not make hiring decisions and it is not trained on historical hiring outcomes.

1.1.5 Objectives

The project has the following objectives. Table 1.1 maps each objective to the module and API endpoint or page that implements it, so that every claim in this report can be verified in the working application.

- **Automate resume parsing:** extract structured information (name, contact details, education, skills, experience, projects) from PDF and DOCX resumes.
- **Score resumes intelligently:** generate a quantitative score from 0 to 100 using weighted criteria.
- **Recommend relevant jobs:** match the candidate profile against job listings with a hybrid NLP model (TF-IDF + Jaccard).
- **Identify skill gaps:** highlight skills that a target job requires but the resume does not contain, and suggest improvements.
- **Provide ATS-compatibility analysis:** assess action-verb usage, keyword density and section completeness.
- **Enable job browsing and filtering:** allow users to explore jobs and filter by attributes such as category, location, job type and experience.
- **Maintain analysis history:** persist and display previous resume analyses for reference.

Table 1.1: Mapping of objectives to implementation

Sr. No.	Objective	Implementing module	API endpoint / page
1	Resume parsing	ResumeParser, SkillExtractor	POST /api/resume/upload; Resume Upload page
2	Resume scoring	ResumeScorer	POST /api/resume/analyze; Resume Analysis page
3	Job recommendation	TFIDFEngine, SimilarityCalculator, RecommendationEngine	POST /api/recommendations; Job Recommendations page
4	Skill-gap analysis	JobMatcher (set difference of job skills and resume skills)	Job Matching page
5	ATS analysis	ResumeScorer (action verbs, keyword density)	POST /api/resume/analyze; Resume Improvement page
6	Job browsing / filtering	Job routes	GET /api/jobs, GET /api/jobs/<id>; Job Recommendations and Job Details pages
7	Analysis history	History routes, AnalysisHistory table	GET /api/history; Analysis History page

2 Literature Survey

2.1 Survey of Existing/Similar System

Automated resume screening and job matching sit at the intersection of information retrieval, text mining and recommender systems. This section surveys the main families of approaches that existing and similar systems are built on, using the sources listed in the References, and explains which of them this project adopts.

2.1.1 Keyword-Based ATS Filtering

Most organisations use Applicant Tracking Systems that filter resumes by keywords, structure and formatting before a recruiter reads them. These systems are fast, but they give the candidate no explanation: a resume can be rejected because of poor formatting, missing sections or an absent keyword, and the candidate cannot tell which one caused it.

2.1.2 Term-Weighting and Vector-Space Similarity

The vector-space model and term weighting described by Salton and McGill [9] represent documents as weighted term vectors, so that the similarity of two texts can be computed as the cosine of the angle between their vectors. TF-IDF is the standard weighting scheme, and an efficient implementation is available in scikit-learn [1]. The method needs no training data and is explainable, but it compares vocabulary rather than meaning.

2.1.3 Set-Similarity Measures

The Jaccard index [10] measures the overlap between two sets as the size of their intersection divided by the size of their union. Applied to the skill sets of a resume and a job, it focuses only on explicit skill matches, which are what recruiters ultimately check. It ignores all text that is not part of the skill list.

2.1.4 Ontology-Driven Resume Information Extraction

Celik and Elci [11] propose a semantic-based approach to resume information extraction in which domain knowledge guides the recognition of resume content. This project follows the same idea in a lightweight form: a curated skills taxonomy with aliases resolves different spellings (for example “reactjs” and “React”) to one canonical skill.

2.1.5 Recommender Systems

Lu et al. [12] survey the development of recommender systems and their application areas. Content-based methods recommend items whose features match a user profile, collaborative methods use the behaviour of similar users, and hybrid methods combine approaches. Because this project has no application-outcome data, it uses a content-based hybrid of TF-IDF similarity and skill overlap.

2.1.6 Embedding and Deep-Learning Approaches

Embedding models such as BERT and Sentence-BERT capture semantic similarity (for example that "model deployment" and "MLOps" are related), which TF-IDF cannot. They need more computing resources and are less transparent, so they are kept as future work (Section 3.6).

Table 2.1: Comparison of existing approaches

Approach	Strength	Limitation	Use in this project
Keyword-based ATS filtering	Fast, widely deployed	Opaque to candidates; sensitive to formatting	ATS-compatibility analysis (action verbs, keyword density, completeness)
TF-IDF + cosine similarity [1][9]	No training data; explainable	Compares vocabulary, not meaning	Text similarity, weight 0.55
Jaccard skill overlap [10]	Reflects hard skill requirements	Ignores text outside the skill list	Skill similarity, weight 0.45
Ontology-driven extraction [11]	Resolves aliases to canonical skills	Needs a maintained taxonomy	skills_taxonomy.json with seven categories
Recommender systems [12]	Ranks items for a user profile	Collaborative methods need outcome data	Content-based hybrid ranking
BERT / Sentence-BERT embeddings	Semantic matching	Higher cost; less transparent	Future work

2.1.7 Gap Addressed by This Project

The approaches above are usually applied separately. This project integrates parsing, taxonomy-based skill extraction, transparent weighted scoring, hybrid job recommendation and skill-gap analysis in one application that runs on ordinary hardware with no training data. Every score is a visible weighted sum or a similarity value, so the user can see exactly which part of the resume to improve.

3 Proposed System

3.1 Introduction

3.1.1 Workflow Overview

The end-to-end flow is: upload the resume, validate the file, extract text, detect sections, extract structured fields, extract skills, compute the resume score, compute hybrid similarity against all jobs, rank the jobs, compute skill gaps and present the results. Section 3.3 describes each of the five phases in detail.

3.1.2 Relevance to Data Warehousing and Mining

Although the application is presented as a web system, its core is a data-mining pipeline: raw, unstructured data (resumes and job descriptions) is collected, cleaned, transformed into numeric features, compared with similarity measures and presented as ranked knowledge. Table 3.1 relates each stage of the project to the corresponding topic of the Data Warehouse and Mining subject.

Table 3.1: Mapping of project stages to DWM concepts

DWM concept	Where it is used in this project
Data collection and integration (ETL)	Data is extracted from three sources (uploaded resumes, jobs.csv, skills_taxonomy.json), transformed (text extraction, regex normalisation, alias resolution) and loaded into the SQLite database and in-memory cache
Data cleaning and pre-processing	Lower-casing, tokenisation and stop-word removal with NLTK [5]; extraction of missing location, job type and experience values from job descriptions with regular expressions
Feature extraction and selection	TF-IDF vectorisation with unigrams and bigrams, limited to the 5000 most informative features [1]
Text mining	Section detection, entity extraction (name, email, phone, degree) and skill extraction from unstructured resume text
Similarity and distance measures	Cosine similarity on TF-IDF vectors and the Jaccard index on skill sets [9][10]
Ranking and recommendation	Hybrid score used to rank all jobs for a resume and return the top-N matches [12]
Knowledge presentation	Dashboard that summarises scores, skill categories and history, and shows patterns such as skill-category counts and skill gaps

3.1.3 Dataset Description

Job Listings Dataset (jobs.csv)

The primary dataset is a curated CSV file of real-world job postings from the technology sector. Each record contains the fields shown in Table 3.2. The file is read with Pandas and cached in memory by the RecommendationEngine so that ranking is fast.

Table 3.2: Fields of the job dataset

Field	Description
id	Unique job identifier
title	Job title (for example Data Scientist, Backend Developer)
company	Hiring organisation
location	Job location (city, country or remote)
description	Full job description text, used for TF-IDF
skills	Required skills (comma-separated), used for Jaccard overlap and skill-gap analysis
experience	Required years of experience
salary	Compensation range
job_type	Employment type (Full-time, Internship, Contract, Remote, Hybrid)
category	Job domain (for example AI/ML, Backend, Frontend)
link	Source URL of the job posting

Because some postings do not provide clean values for location, job type or experience, the RecommendationEngine additionally extracts these attributes from the raw description text using regular expressions. This data-cleaning step makes filtering more reliable on noisy data.

Skills Taxonomy (skills_taxonomy.json)

A curated JSON taxonomy organises hundreds of skills into seven domains. Each skill can have aliases (for example “reactjs” maps to “React” and “sklearn” maps to “Scikit-learn”), so that different spellings in a resume are resolved to one canonical skill name.

Table 3.3: Skill taxonomy categories

Category	Examples
programming_languages	Python, Java, C++, JavaScript, TypeScript, Go, Rust
frameworks_and_libraries	Flask, Django, React, Node.js, Pandas, Scikit-learn
databases	MySQL, PostgreSQL, MongoDB, SQLite, Redis
cloud_and_devops	AWS, Docker, Kubernetes, CI/CD, GitHub Actions
tools_and_platforms	Postman, Power BI, Tableau, VS Code
concepts_and_domains	Machine Learning, NLP, Deep Learning, REST API
soft_skills	Communication, Leadership, Problem Solving

Resume Data (User-Uploaded)

Candidate resumes are uploaded by the user in PDF or DOCX format with a maximum size of 10 MB. Uploaded files are validated for type and size, parsed, and the resulting structured data is stored in the database. Resumes are personal data; the application stores them only for the purpose of analysis and history, and a production deployment would need explicit consent, encryption and a deletion policy (see Section 3.5.4, Limitations).

3.2 Architecture/ Framework

3.2.1 System Architecture

The system follows a client-server architecture. The React single-page application communicates with the Flask backend over a REST API using JSON [2][6]. The backend is organised into routes, services, an ML layer and a persistence layer. Figure 3.1 shows the architecture and Table 3.4 lists the components of each layer.

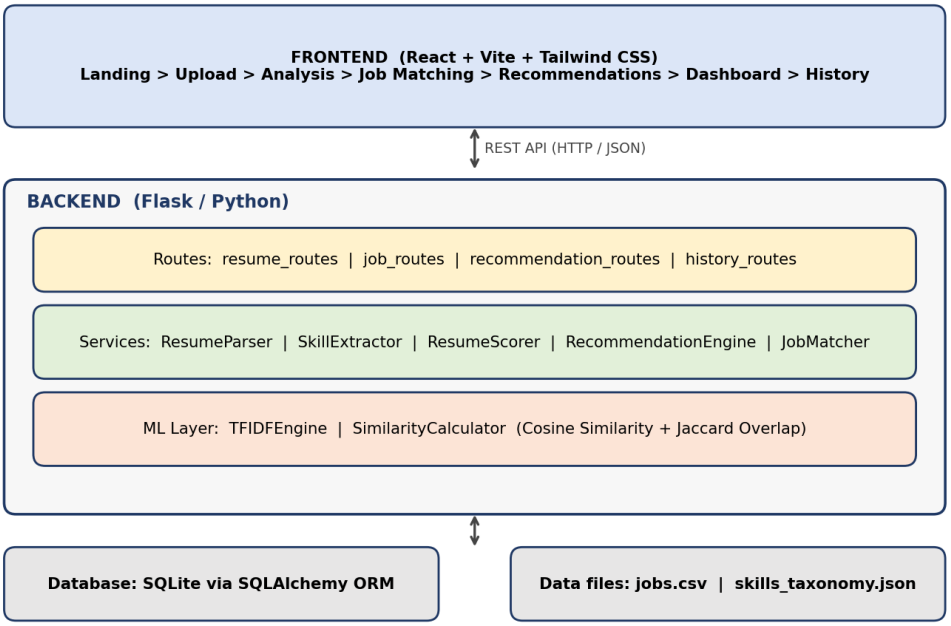


Figure 3.1: System architecture

Table 3.4: Layered architecture

Layer	Components
Frontend	React + Vite + Tailwind CSS. Pages: Landing, Resume Upload, Resume Analysis, Job Recommendations, Job Matching, Job Details, Dashboard, Resume Improvement, Analysis History, About
API layer	Flask routes: resume_routes, job_routes, recommendation_routes, history_routes
Service layer	ResumeParser, SkillExtractor, ResumeScorer, RecommendationEngine, JobMatcher
ML layer	TFIDFEngine, SimilarityCalculator (cosine similarity and Jaccard overlap)
Data layer	SQLite through SQLAlchemy ORM [8]; jobs.csv and skills_taxonomy.json loaded at start-up

3.2.2 Database Design

Application data is stored in SQLite through the SQLAlchemy ORM. The data model has four entities, summarised in Table 3.5. Job postings themselves are read from jobs.csv and cached in memory for ranking; the Job entity describes the structure of a posting as exposed by the job API endpoints.

Table 3.5: Database entities

Entity	Purpose	Typical fields
Resume	Stores the uploaded file reference and parsed content	id, filename, raw text, parsed fields (name, email, phone, education, skills), upload time
ResumeAnalysis	Stores the score of one analysis run	id, resume id, overall score, six component scores, strengths, weaknesses, recommendations
Job	Represents a job posting	id, title, company, location, skills, category, job type, link
AnalysisHistory	Log of past analyses shown to the user	id, resume id, analysis id, timestamp, summary

3.3 Algorithm and Process Design

3.3.1 Phase 1: Resume Ingestion and Parsing

- The user uploads a PDF or DOCX file through the drag-and-drop interface.
- The server validates the file extension and the 10 MB size limit and sanitises the filename using Werkzeug.
- **Text extraction:** pdfplumber [3] is the primary PDF extractor because it preserves reading order well. If it returns little or no text, the pypdf library is used as a fallback. DOCX files are read with python-docx [4], including paragraphs and table cells.
- **Section detection:** regular expressions search for common headings (Education, Experience, Projects, Skills, Contact and their synonyms) and split the text into sections.
- **Structured extraction:** name, email and phone are found by pattern matching; education is detected using 18 degree patterns (for example B.Tech, M.Tech, MBA, Ph.D.); work entries and projects are counted and stored with their descriptions.

3.3.2 Phase 2: Skill Extraction

- The taxonomy file with seven categories is loaded once and converted into a lookup map from every alias and canonical name to its canonical skill, following the idea of ontology-driven resume extraction [11].
- Resume text is lower-cased and each lookup key is searched with a regular expression using word boundaries, so that, for example, “Java” is not matched inside “JavaScript”.

- The output is a dictionary of skills grouped by category plus a flat list of unique skills. The flat list is used for Jaccard similarity and the grouped dictionary is used to measure category diversity in scoring.

3.3.3 Phase 3: Resume Scoring (Weighted ATS Model)

The overall score is a weighted sum of six component scores, each normalised to the range 0-100:

$$Overall = 0.35 \times Skills + 0.20 \times Experience + 0.15 \times Projects + 0.10 \times Education + 0.10 \times Completeness + 0.10 \times Keywords$$

Table 3.6: Scoring components

Component	Weight	How it is computed	Why it matters
Skills Match	35%	Number of unique skills found (saturating at 15) combined with category diversity (up to 5 of 7 domains)	Skills are the strongest signal of fit for technical roles, so this has the highest weight
Experience	20%	Number of work entries and the depth of responsibilities described per role	Shows practical exposure and progression
Projects	15%	Number of projects; three or more earns 95/100	Important for freshers who lack work experience
Education	10%	Degree level: Ph.D. > Master's > Bachelor's > Diploma	Baseline qualification
Completeness	10%	Presence of name, email, phone and all expected sections	Incomplete resumes are hard for ATS and recruiters to process
Keyword Relevance	10%	Count of 24+ ATS action verbs (developed, built, implemented, optimized, deployed, automated, led, analyzed, ...) and keyword density	Action verbs and relevant keywords improve ATS pass rates

The weights were chosen empirically: skills and experience receive the largest share because they are the primary signals recruiters use for technical roles, while education, completeness and keywords act as supporting signals. The weights are stored in a configuration object, so they can be changed for a different industry without modifying the scoring logic.

From these components the scorer also generates strengths (components above a threshold), weaknesses (components below a threshold) and concrete recommendations such as "add at least two more projects" or "use more action verbs at the start of bullet points".

3.3.4 Phase 4: Hybrid Job Recommendation

Each job is compared with the candidate using two complementary measures.

TF-IDF Cosine Similarity

Resume text and job text (title, description and skills) are pre-processed with NLTK (lower-casing, tokenisation, stop-word removal) and converted to TF-IDF vectors [9] using scikit-learn's TfidfVectorizer [1] with ngram_range = (1, 2), max_features = 5000 and sublinear_tf = True. Unigrams and bigrams allow phrases such as "machine learning" and "rest api" to be treated as single features. Sublinear term frequency (1 + log tf) prevents a term repeated many times from dominating. The similarity of two vectors a and b is:

$$cosine(a, b) = (a \cdot b) / (||a|| \times ||b||)$$

Skill Jaccard Overlap

Let A be the set of skills extracted from the resume and B the set of skills required by the job. The Jaccard index [10] is:

$$J(A, B) = |A \cap B| / |A \cup B|$$

Unlike TF-IDF, which compares all words, Jaccard focuses only on explicit skill matches, which are what recruiters ultimately check.

Hybrid Score and Scaling

$$Hybrid = 0.55 \times Cosine_TFIDF + 0.45 \times Jaccard_Skills$$

TF-IDF receives the slightly higher weight because the full text carries context (domain, responsibilities, tools) that a skill list alone cannot. Jaccard still receives a large weight so that a job whose required skills are mostly missing from the resume cannot rank high on vocabulary similarity alone. The 0.55/0.45 split was selected empirically by inspecting the rankings produced for sample resumes.

Raw hybrid values tend to be small (cosine similarity between a resume and a job description rarely exceeds 0.5), which would show users unhelpfully low percentages. A non-linear scaling function therefore maps the hybrid score into the range 15% to 98%, which avoids the extremes of 0% and 100%. If the skill overlap is at least 90%, a small bonus boost is added.

Worked Example

Suppose a candidate has the skills A = {Python, SQL, Flask, Machine Learning, Pandas} and a Data Scientist job requires B = {Python, Machine Learning, Scikit-learn, Pandas, SQL,

Statistics}. The intersection contains four skills (Python, SQL, Machine Learning, Pandas) and the union contains seven. Therefore J = 4 / 7 = 0.571. If the TF-IDF cosine similarity between the resume text and the job description is 0.42 (an illustrative value), then:

$$Hybrid = 0.55 \times 0.42 + 0.45 \times 0.571 = 0.231 + 0.257 = 0.488$$

This raw value of 0.488 is then passed through the non-linear scaling to obtain the final match percentage shown to the user.

Ranking

The RecommendationEngine loads all jobs once, caches them in memory, computes the hybrid score of the resume against every job, sorts in descending order and returns the top-N jobs together with their match percentages, matching skills and missing skills.

3.3.5 Skill-Gap Analysis

For each recommended or selected job, the system computes two sets: matched skills (resume skills ∩ job skills) and missing skills (job skills − resume skills). The missing skills are displayed in the Job Matching page and are used to produce suggestions such as "learn Scikit-learn and Statistics to improve your match for this role." In the example above, the matched skills are Python, SQL, Machine Learning and Pandas, and the missing skills are Scikit-learn and Statistics.

3.3.6 Phase 5: Results and Presentation

- Overall score (0-100) with a chart and breakdown by component.
- ATS-compatibility report with action-verb analysis and section completeness.
- Strengths, weaknesses and recommendations in plain language.
- Top-N ranked job recommendations with match percentages and skill gaps.
- Every analysis is saved to the history table so the user can revisit it later.

3.4 Details of Hardware & Software

3.4.1 Hardware Requirements

The system uses TF-IDF and Jaccard similarity, which need no training data or GPU, so it runs on an ordinary personal computer. Table 3.7 lists the configuration recommended for running the application locally.

Table 3.7: Hardware requirements

Component	Requirement
Processor	Dual-core 64-bit processor or better
Memory (RAM)	4 GB minimum (8 GB recommended)
Storage	About 1 GB of free disk space for dependencies, dataset and database
GPU	Not required
Network	Needed only to install dependencies; the application runs locally (frontend on port 5173, backend on port 5000)

3.4.2 Software Requirements

Backend

Table 3.8: Backend technologies

Technology	Version	Purpose
Python	3.11+	Core backend language
Flask	3.0.0	REST API web framework
Flask-CORS	6.0.2	Cross-Origin Resource Sharing between frontend and backend
SQLAlchemy	2.0.45	ORM for database interaction
Flask-SQLAlchemy	3.1.1	Flask integration for SQLAlchemy
SQLite	Built-in	Lightweight relational database

Machine Learning and NLP

Table 3.9: Machine learning and NLP libraries

Technology	Version	Purpose
Scikit-learn	1.8.0	TF-IDF vectoriser and cosine similarity
NLTK	3.10.3	Text pre-processing and tokenisation
NumPy	2.2.6	Numerical computation and array operations
Pandas	2.2.3	CSV parsing and data manipulation

Resume Parsing and Document Libraries

Table 3.10: Parsing and document libraries

Technology	Version	Purpose
pdfplumber	0.11.10	Primary PDF text extraction
pypdf	6.19.0	Fallback PDF reading
python-docx	1.2.0	DOCX text extraction
ReportLab	4.4.10	PDF report generation
Werkzeug	3.0.1	Secure file upload handling

Frontend

Table 3.11: Frontend technologies

Technology	Purpose
React 18+	Component-based user interface
Vite	Fast build tool and development server
React Router v6	Client-side routing for the single-page application
Tailwind CSS v3	Utility-first styling and responsive design
Axios / Fetch API	HTTP communication with the REST API

3.4.3 Why These Technologies?

- Flask is lightweight and integrates naturally with the Python ML ecosystem, so the model code and the API run together.
- SQLite needs no separate server, which keeps setup simple; SQLAlchemy makes a later move to PostgreSQL straightforward.
- TF-IDF and Jaccard need no training data or GPU, are fast enough to rank many jobs per request, and give explainable scores.
- React with Vite gives fast development and a responsive interface, and Tailwind CSS speeds up consistent styling.

3.5 Experiment and Results

3.5.1 Output Screenshots

The following screenshots show the working application running locally (React frontend on port 5173, Flask backend on port 5000).

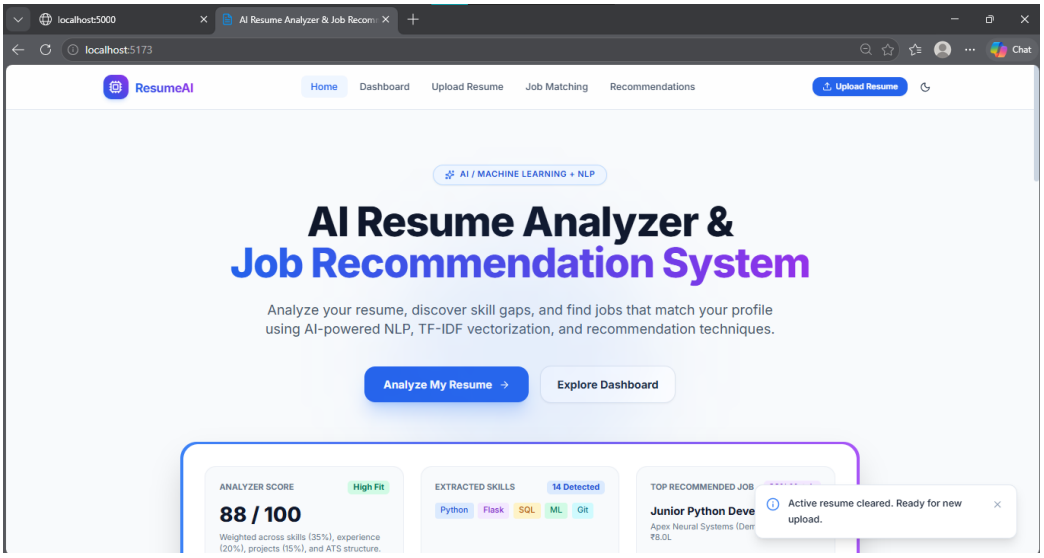


Figure 3.2: Landing page with the platform introduction and a sample score card

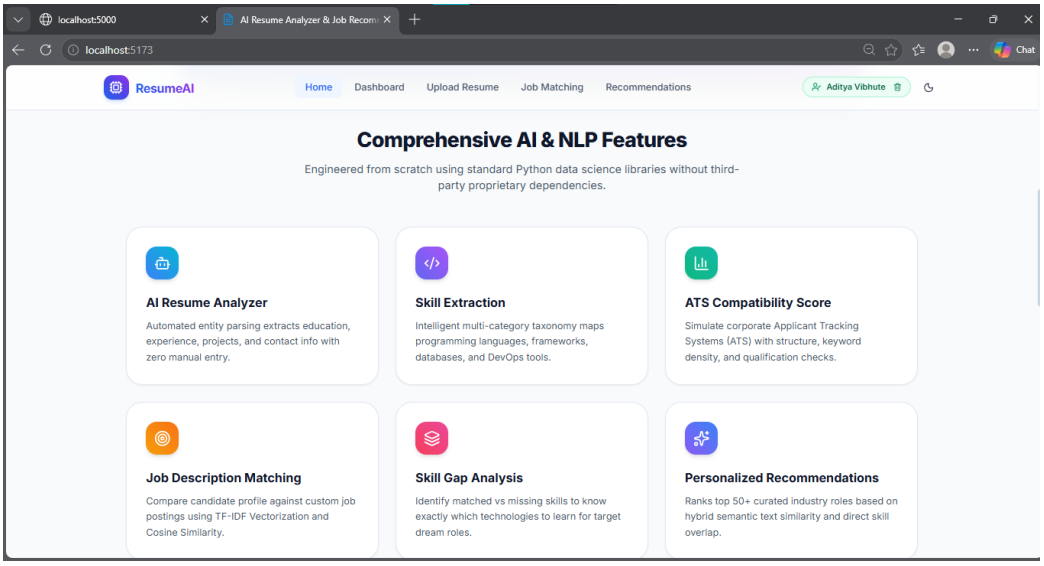


Figure 3.3: Landing page: comprehensive AI and NLP features

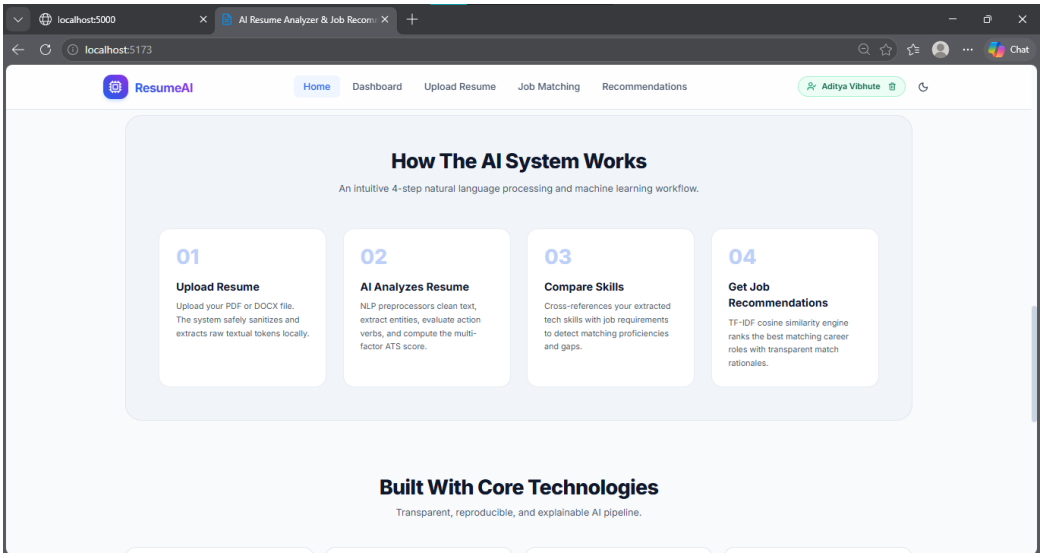


Figure 3.4: Landing page: how the AI system works (four-step workflow)

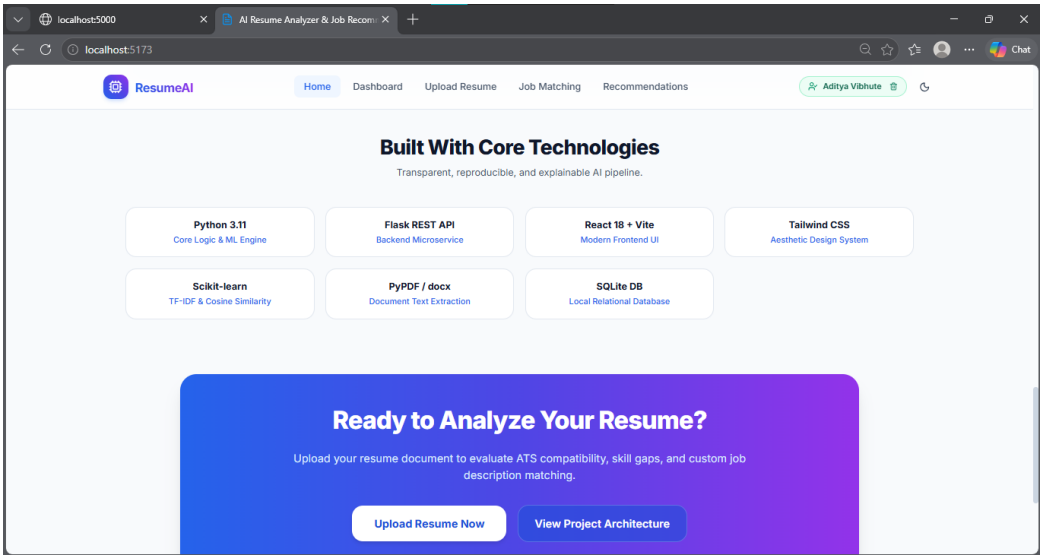


Figure 3.5: Landing page: core technologies and call to action

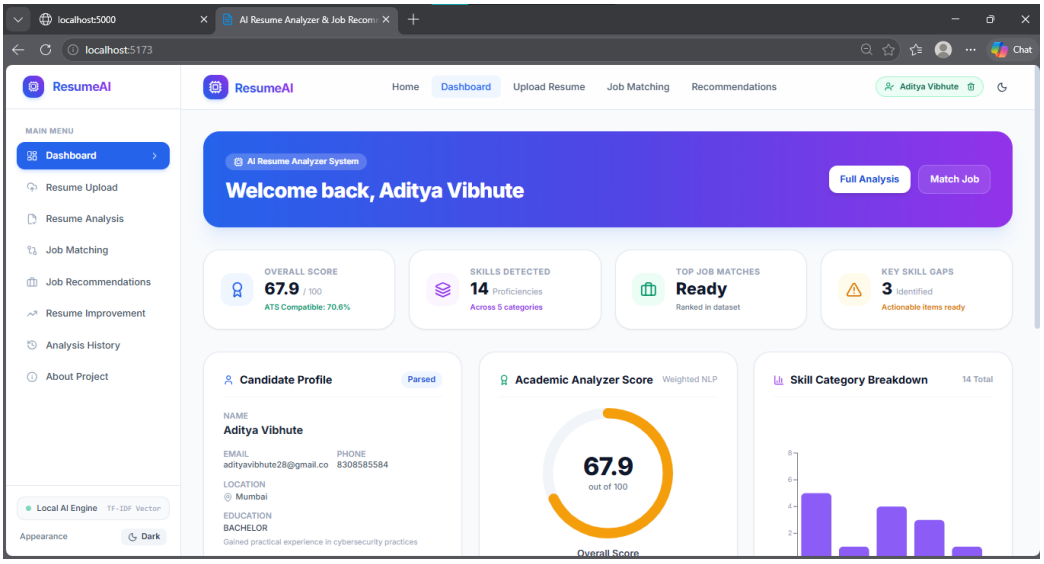


Figure 3.6: Dashboard: overall score, skills detected, key skill gaps and candidate profile

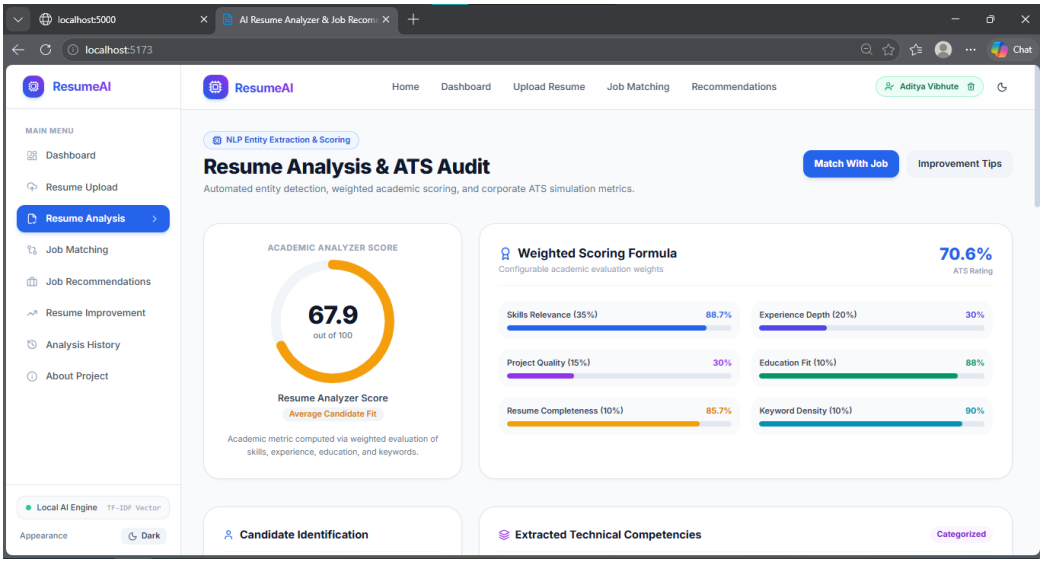


Figure 3.7: Resume Analysis: analyser score and weighted scoring formula

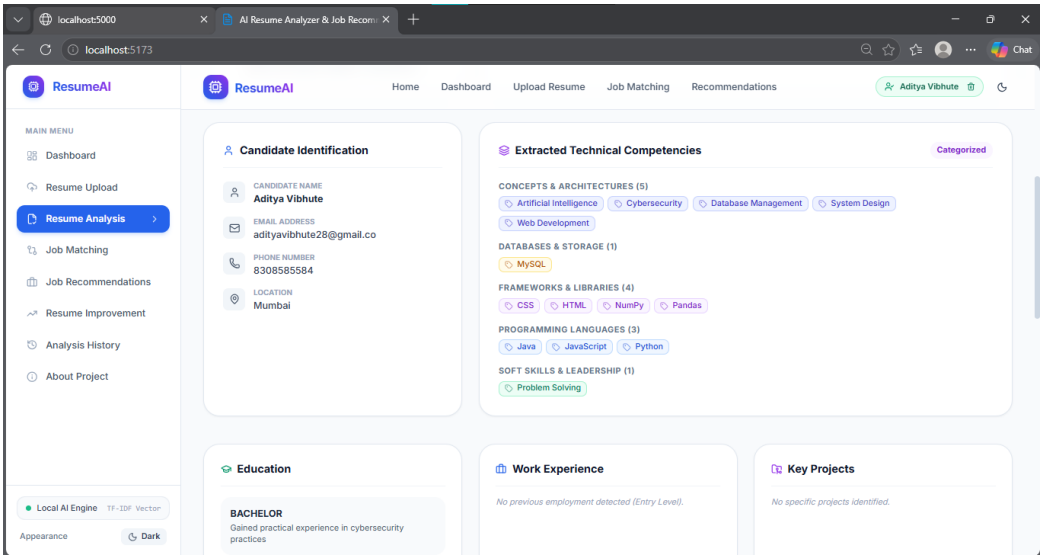


Figure 3.8: Resume Analysis: candidate identification and extracted technical competencies

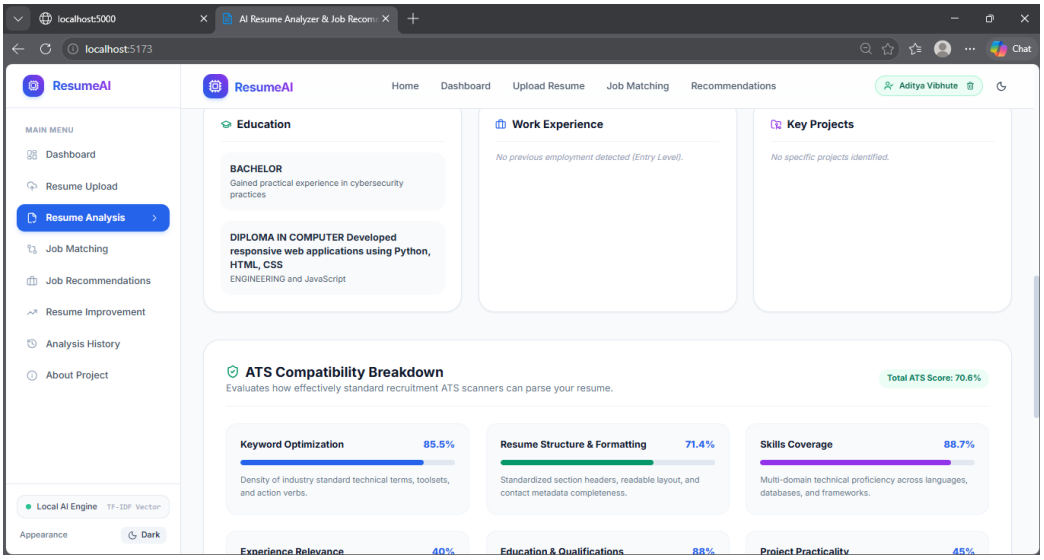


Figure 3.9: Resume Analysis: education, work experience, projects and ATS compatibility breakdown

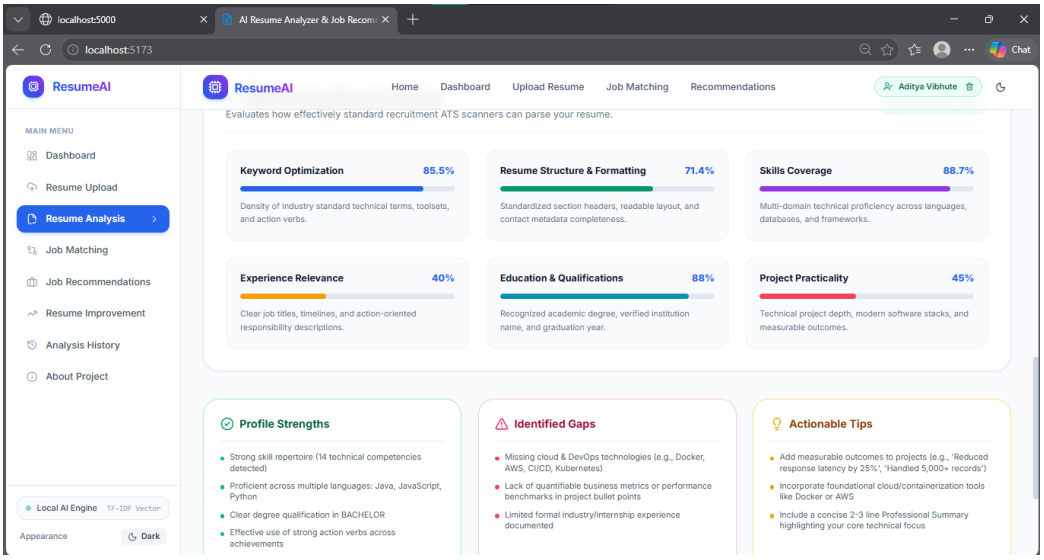


Figure 3.10: Resume Analysis: quality metrics, strengths, identified gaps and actionable tips

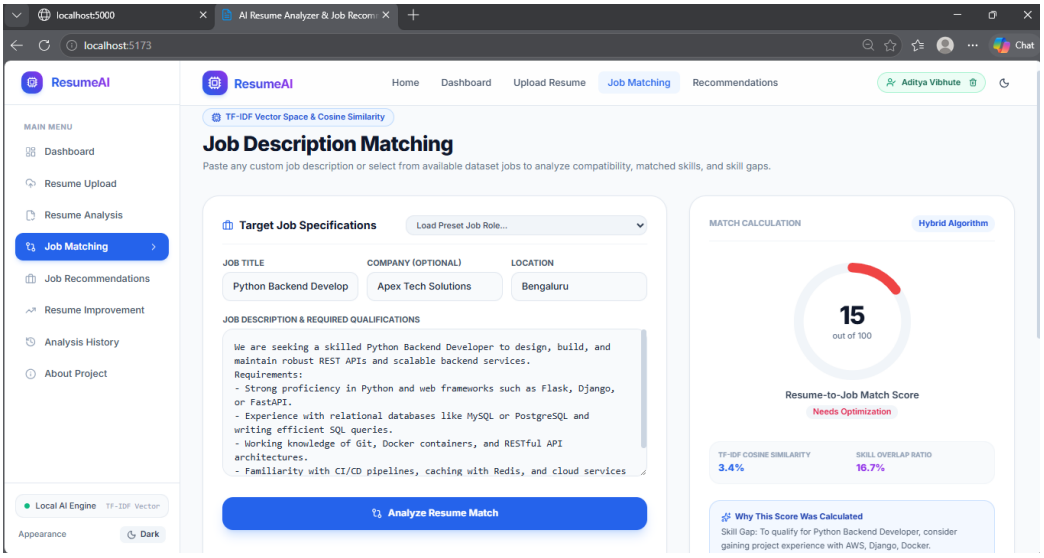


Figure 3.11: Job Matching: job description input and match calculation

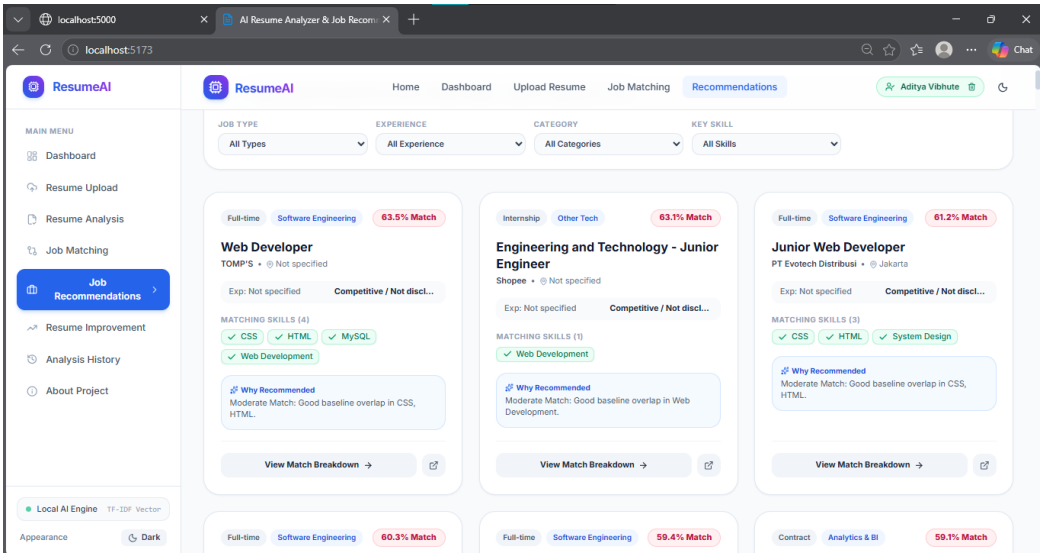


Figure 3.12: Job Recommendations: AI-ranked jobs with match percentages and filters

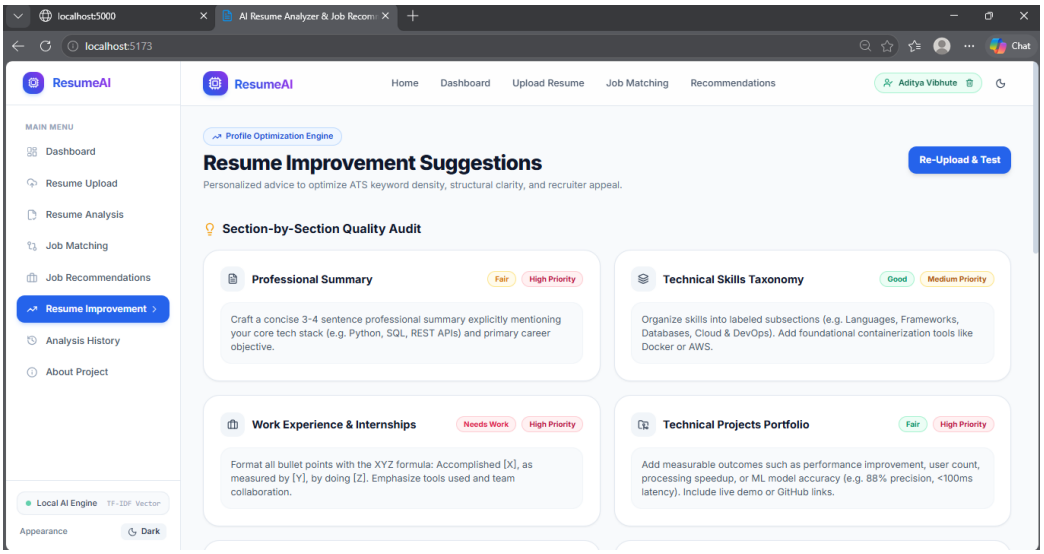


Figure 3.13: Resume Improvement: section-wise quality audit and suggestions

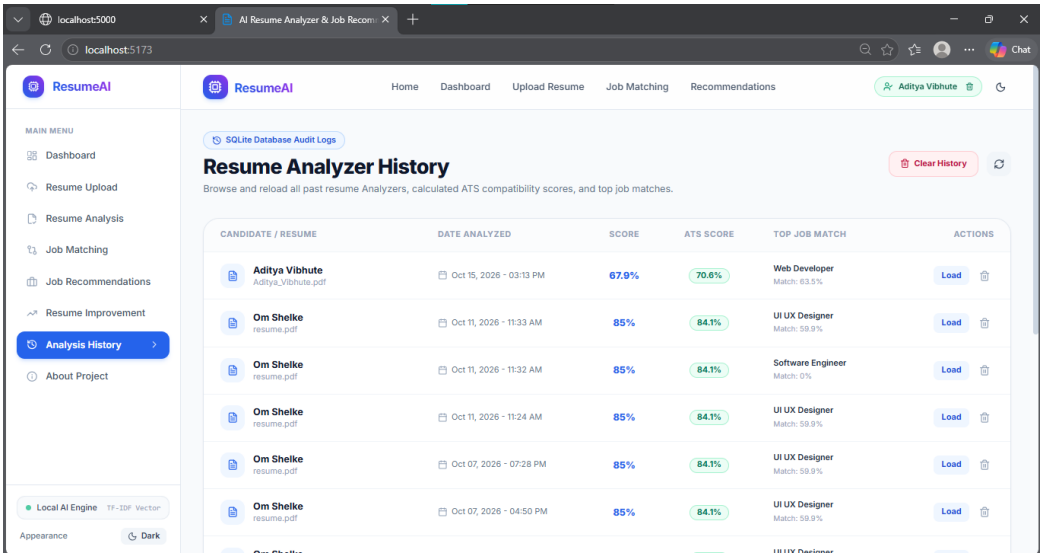


Figure 3.14: Analysis History: previous analyses with scores and top job match

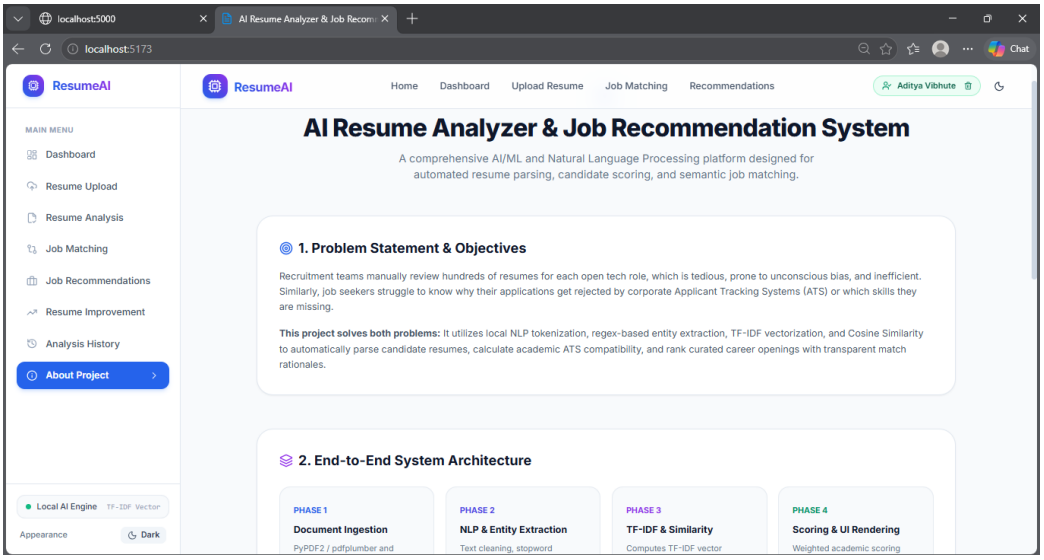


Figure 3.15: About Project: problem statement and system architecture

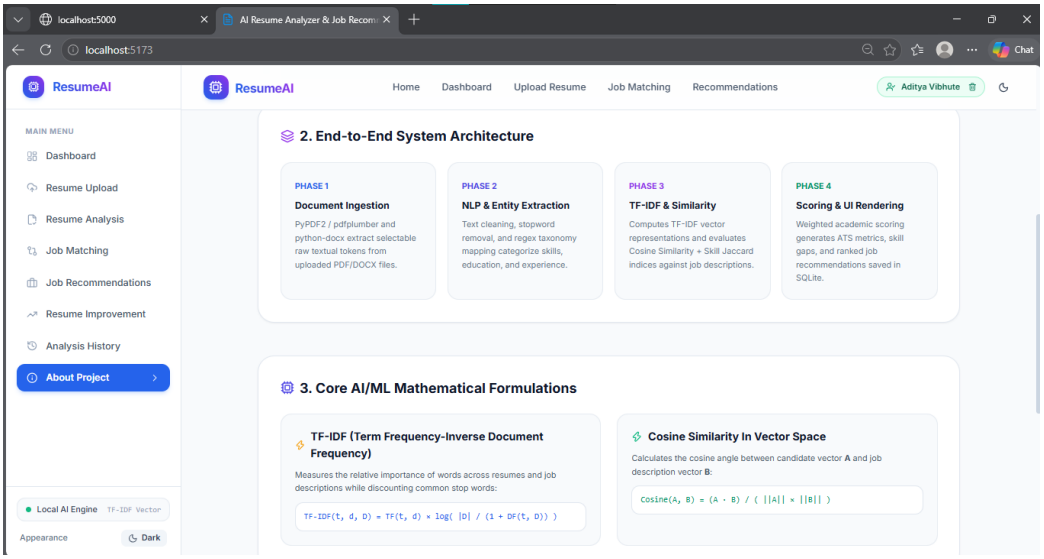


Figure 3.16: About Project: end-to-end workflow and mathematical formulations

3.5.2 Sample Scoring Breakdown

Table 3.12 shows how three representative candidate profiles are scored. The totals can be verified with the weights in Section 3.3.3; for example, for the Fresh Graduate: 0.35 × 70 + 0.20 × 45 + 0.15 × 95 + 0.10 × 85 + 0.10 × 95 + 0.10 × 75 = 24.5 + 9 + 14.25 + 8.5 + 9.5 + 7.5 = 73.25.

Table 3.12: Sample scoring breakdown (scenario profiles)

Scenario	Skills	Exp.	Proj.	Edu.	Compl.	Keyw.	Total
Fresh Graduate (strong)	70	45	95	85	95	75	73.25
Mid-Level Developer	90	90	82	80	100	85	88.3
Senior Professional	100	90	95	90	100	90	95.25

3.5.3 Discussion

The screenshots confirm that all stages of the pipeline work together. In the test run shown in Figures 3.6 to 3.10, the uploaded resume was parsed successfully, 14 skills were detected across 5 categories, the overall resume score was 67.9 out of 100, the ATS compatibility score was 70.6% and 3 key skill gaps were identified. The same analysis is then available in the Analysis History page and can be reloaded.

The scores follow the expected behaviour of the model: resumes with more evidence (skills, projects, experience, complete sections) score higher, while missing sections lower the corresponding component and produce a specific recommendation. Because every score is a weighted sum of visible components, a user can see exactly which part of the resume to improve, which is an advantage over black-box screening tools.

The hybrid recommendation combines two complementary signals. TF-IDF captures term-weighted textual similarity across the whole resume and job description, including context words and domain vocabulary. Jaccard captures exact skill overlap, which reflects hard requirements. Using both reduces the weaknesses of each: TF-IDF alone can rank a job highly because of shared general vocabulary even when key skills are missing, while Jaccard alone ignores everything that is not in the taxonomy. Neither method captures deep semantic similarity (for example that "model deployment" and "MLOps" are related), which is why embedding-based models are listed as future work.

3.5.4 Limitations

- Regex-based parsing is sensitive to layout: unusual templates, multi-column layouts, tables and text inside images can reduce extraction quality.
- Scanned (image-only) PDFs are not supported because the system has no OCR step.
- The taxonomy is limited to technology skills, so resumes from other domains will not be scored well.
- TF-IDF and Jaccard do not understand synonyms or related concepts unless they are listed as aliases in the taxonomy.
- The scoring weights and the 0.55/0.45 hybrid split are chosen empirically and not learned from hiring outcomes, so the score estimates resume strength and not the probability of being hired.
- The job dataset is static, so recommendations are only as current and comprehensive as jobs.csv.
- No formal bias audit has been performed. Features such as the number of work entries or degree level could disadvantage some candidates.
- Privacy: the prototype stores data in a local SQLite file without encryption, authentication or a retention policy.
- Only English-language resumes are supported, and the evaluation was done on a small number of sample resumes.

3.6 Conclusion and Future Work

3.6.1 Conclusion

This project demonstrates the development of a functional AI Resume Analyzer & Recommendation system that addresses real-world recruitment challenges. It combines resume parsing, skill extraction, transparent scoring, hybrid job recommendation, skill-gap analysis and a modern web interface in a single full-stack application, and it applies core Data Warehouse and Mining concepts such as data pre-processing, text mining, feature extraction, similarity measures and ranking.

Key achievements are:

- Automated resume parsing for PDF and DOCX with a multi-library fallback for robust text extraction.

- Structured extraction of education, skills, experience and contact information using regular expressions and a curated taxonomy.
- A transparent and configurable six-component weighted scoring system.
- A hybrid recommendation model (0.55 TF-IDF + 0.45 Skill Jaccard) that produces ranked job matches with explainable scores.
- Skill-gap analysis that tells users which skills they should add for a target job.
- A full-stack implementation with a React frontend, Flask REST API and SQLite persistence.

The hybrid approach is effective because pure keyword matching ignores the weighting and context of terms, while pure TF-IDF can overlook explicit skill requirements; combining term-weighted text similarity with exact skill overlap handles both situations without deep-learning infrastructure. The project works as an academic proof of concept and as a foundation that can be extended into a production-grade recruitment platform by adding the improvements described below.

3.6.2 Future Work

Deep learning integration. Replace TF-IDF with BERT or Sentence-BERT embeddings for stronger semantic matching, and use spaCy Named Entity Recognition (NER) for more accurate extraction of names, organisations, dates and skills.

Advanced recommendation. Add collaborative filtering based on application outcomes and Learning-to-Rank (LTR) models trained on recruiter feedback to improve precision.

Real-time job data. Integrate live job feeds from sources such as LinkedIn, Indeed or Naukri (subject to their terms of use and API access) and run scheduled crawlers with Celery and Redis.

OCR support. Add Tesseract or a cloud OCR service so that scanned resumes can be processed.

Multi-language support. Extend the parser to Hindi, Marathi, Tamil and other Indian languages using multilingual NLP models.

Generative AI features. Use a large language model such as Google Gemini or GPT-4 to generate personalised resume improvement suggestions and draft cover letters, while keeping the transparent scores as the base.

Recruiter dashboard. Build a recruiter portal where HR teams can post jobs, batch-upload resumes and automatically shortlist candidates.

Authentication and privacy. Add user accounts, consent management, encryption at rest and a data-deletion option.

Fairness auditing. Add tests and monitoring for bias in the scoring and recommendation pipeline.

Cloud deployment. Deploy on GCP or AWS with PostgreSQL, Redis caching, Docker containers and Kubernetes auto-scaling.

Mobile application. Develop a companion Android / iOS application using React Native for on-the-go resume analysis.

References

[1] Scikit-learn Documentation: TF-IDF Vectorizer and Cosine Similarity. https://scikit-learn.org/stable/

[2] Flask Documentation: Web Framework for Python. https://flask.palletsprojects.com/en/3.0.x/

[3] pdfplumber: Extracting text from PDFs. https://github.com/jsvine/pdfplumber

[4] python-docx: Reading and writing DOCX files. https://python-docx.readthedocs.io/

[5] NLTK: Natural Language Processing Toolkit. https://www.nltk.org/

[6] React Documentation: Component-Based UI. https://react.dev/

[7] Vite: Next Generation Frontend Tooling. https://vitejs.dev/

[8] SQLAlchemy: The Python SQL Toolkit and ORM. https://docs.sqlalchemy.org/

[9] Salton, G., and McGill, M. J. (1983). Introduction to Modern Information Retrieval. McGraw-Hill.

[10] Jaccard, P. (1901). Etude comparative de la distribution florale. Bulletin de la Societe Vaudoise des Sciences Naturelles, 37, 547-579.

[11] Celik, D., and Elci, A. (2013). A semantic-based approach for resume information extraction. IEEE INISTA.

[12] Lu, J. et al. (2015). Recommender system application developments: A survey. Decision Support Systems, 74, 12-32.